

Plan/Build 模式端到端流程

概述

Plan/Build 是 oh-my-opencode 的核心编排模式，将复杂任务分为两个阶段：

1. **Plan 阶段** — Prometheus 生成详细的实施计划
2. **Build 阶段** — Sisyphus/Atlas 按计划并行执行

涉及 4 个 Agent 协作：**Metis**（预分析）→ **Prometheus**（生成计划）→ **Momus**（审查）→ **Sisyphus/Atlas**（执行）

Plan 阶段

Prometheus 系统 Prompt 组装

源文件：src/agents/prometheus/system-prompt.ts (69 行)

Prometheus 的系统 prompt 由 6 个模块化片段拼接而成：

```
// src/agents/prometheus/system-prompt.ts
import { PROMETHEUS_IDENTITY_CONSTRAINTS } from "../identity-constraints" // 337 行
import { PROMETHEUS_INTERVIEW_MODE } from "../interview-mode" // 332 行
import { PROMETHEUS_PLAN_GENERATION } from "../plan-generation" // 220 行
import { PROMETHEUS_HIGH_ACCURACY_MODE } from "../high-accuracy-mode" // 64 行
import { PROMETHEUS_PLAN_TEMPLATE } from "../plan-template" // 328 行
import { PROMETHEUS_BEHAVIORAL_SUMMARY } from "../behavioral-summary" // 80 行

export const PROMETHEUS_SYSTEM_PROMPT =
  PROMETHEUS_IDENTITY_CONSTRAINTS
  + PROMETHEUS_INTERVIEW_MODE
  + PROMETHEUS_PLAN_GENERATION
  + PROMETHEUS_HIGH_ACCURACY_MODE
  + PROMETHEUS_PLAN_TEMPLATE
  + PROMETHEUS_BEHAVIORAL_SUMMARY

export const PROMETHEUS_PERMISSION = { edit: "allow", bash: "allow", webfetch: "allow",

export type PrometheusPromptSource = "default" | "gpt" | "gemini"

export function getPrometheusPromptSource(model?: string): PrometheusPromptSource {
  // 根据模型名判断 prompt 来源: GPT-5.2 / Gemini / Claude
}

export function getPrometheusPrompt(model?: string): string {
  // 返回对应模型优化的 prompt
}
```

触发条件

当 Sisyphus 在 Phase 0 Intent Gate 中检测到以下情况时委派给 Prometheus:

- 复杂开放性需求
- 多步骤实现任务
- 需要架构评估的任务
- 用户显式请求规划

Phase 1: Interview（需求访谈）

源文件: src/agents/prometheus/interview-mode.ts

Step 0: 意图分类

- └─ Trivial/Simple → 直接生成计划（跳过面谈）
- └─ Refactoring → 收集当前行为、测试、影响范围
- └─ Build from Scratch → 先 explore 现有模式再提问
- └─ Mid-sized Task → 精确界限、防 AI 偏离
- └─ Collaborative → 频繁检查点设计
- └─ Architecture → 战略分析 + oracle 咨询
- └─ Research → 退出标准、并行探查策略

Step 1: 预研

- └─ explore agents (background) → 搜索代码库模式
- └─ librarian agents (background) → 搜索外部文档

Step 2: 面谈循环

每轮结束自动运行"清关检查"(6 项):

1. ☒ 所有需求已明确?
2. ☒ 技术约束已识别?
3. ☒ 验收标准可定义?
4. ☒ 依赖关系已映射?
5. ☒ 风险已评估?
6. ☒ 足够开始规划?

- 全部 YES → 自动进入 Phase 2
- 任一 NO → 继续面谈

Step 3: 记录

- └─ 持续写入 `.sisyphus/drafts/{name}.md`

Phase 2: Plan Generation (计划生成)

源文件: `src/agents/prometheus/plan-generation.ts` (220 行)

导出: `PROMETHEUS_PLAN_GENERATION` (const string)

```
// src/agents/prometheus/plan-generation.ts - 核心工作流摘录
// 触发条件: Clearance Check 全通过 (自动) 或用户显式触发
// 工作流:
// 1. 触发 → 立即 TodoWrite 注册 plan-1 到 plan-8 共 8 个待办步骤
// 2. Metis 咨询 (必须): task(subagent_type="metis", run_in_background=false, prompt=").
// 3. 生成 .sisyphus/plans/{name}.md (增量写入: Write 骨架 + 多次 Edit 追加任务)
// 4. 自审 gap 分类:
//     - CRITICAL → 必须问用户
//     - MINOR → 自行修复
//     - AMBIGUOUS → 使用默认值
// 5. 呈现摘要 + 用户选择:
//     - "Start Work" → 删除草稿 → 引导 /start-work
//     - "High Accuracy" → 进入 Phase 3 (Momus 审查循环)
```

计划文件模板

源文件: src/agents/prometheus/plan-template.ts

{PLAN_NAME}

TL;DR (≤40 字)

{核心目标一句话描述}

Context

{背景、现有代码状态、依赖关系}

Objectives

{明确的可交付物列表}

Verification Strategy

{QA 方法、测试策略、构建验证}

Parallel Execution Graph

Wave 1 (并行)

Task	Category	Skills	Dependencies	QA
1.1 Setup base structure	quick	[]	none	files exist
1.2 Create UI component	visual-engineering	[tailwind]	none	renders

Wave 2 (依赖 Wave 1)

Task	Category	Skills	Dependencies	QA
2.1 Implement core logic	ultrabrain	[]	1.1, 1.2	tests pass

Wave 3 (集成验证)

Task	Category	Skills	Dependencies	QA
3.1 Integration test	unspecified-high	[testing]	2.1	all green

TODO List (可执行)

Wave 1

- [] Task 1.1: Setup base structure
`task(category="quick", load\_skills=[], description="Setup base", prompt="...")`
- [] Task 1.2: Create UI component
`task(category="visual-engineering", load_skills=["tailwind"], description="UI comp",

Wave 2

- [] Task 2.1: Implement core logic
`task(category="ultrabrain", load\_skills=[], description="Core logic", prompt="...")`

Wave 3

- [] Task 3.1: Integration test

```
`task(category="unspecified-high", load_skills=["testing"], description="Integration"
```

Phase 3: High Accuracy（高精度审查，可选）

源文件: src/agents/prometheus/high-accuracy-mode.ts (64 行, 完整)

导出: PROMETHEUS_HIGH_ACCURACY_MODE (const string)

```
// src/agents/prometheus/high-accuracy-mode.ts - Momus 审查循环核心逻辑摘录
//
// while (true) {
//   1. 提交计划给 Momus:
//     task(subagent_type="momus", run_in_background=false,
//         prompt="Review: .sisyphus/plans/{name}.md")
//
//   2. 检查判定:
//     if (result contains "[OKAY]") break; // 通过审查
//
//   3. [REJECT] → 只关注阻塞性 issue (最多 3 个)
//     → 修复每个 issue → 重新提交
// }
//
// 5 条关键规则:
//   1. 无借口 (Momus 拒绝就修复)
//   2. 修复所有问题 (非部分)
//   3. 持续循环 (无最大重试限制)
//   4. 质量不可妥协
//   5. Momus 调用只传文件路径字符串
//
// "OKAY" 标准:
//   100% 文件引用已验证
//   ≥80% 任务有引用源
//   ≥90% 有验收标准
//   零关键红旗
```

Momus 审查标准

仅检查三项:

1. 引用验证 — 引用的文件/路径是否真实存在
2. 可执行性 — 开发者能否根据描述立即开始工作
3. 关键阻塞 — 是否有矛盾或缺失信息使任务完全无法进行

不检查：方法是否最优、边界情况、验收标准完美度、架构质量

批准偏向：80% 清晰即可通过。默认 [OKAY] 。

防护机制

prometheus-md-only Hook

源文件：src/hooks/prometheus-md-only/hook.ts (87 行)

导出：createPrometheusMdOnlyHook(ctx: PluginInput)

```
// src/hooks/prometheus-md-only/hook.ts — 完整逻辑摘录
export function createPrometheusMdOnlyHook(ctx: PluginInput) {
  return {
    "tool.execute.before": async (input, output) => {
      // 1. 通过 getAgentFromSession 获取当前 agent
      //    → 非 Prometheus 直接放行

      // 2. task/call_omo_agent 工具：
      //    → 注入只读规划警告到 prompt

      // 3. 受阻工具 (BLOCKED_TOOLS: write/edit/apply_patch 等):
      //    → 检查文件路径：
      //        if (!targetPath.startsWith(".sisyphus/") || !targetPath.endsWith(".md"))
      //        throw new Error("Prometheus can only write to .sisyphus/*.md")
      //    }
      //    → .sisyphus/plans/ 路径 → 注入 workflow 提醒
      //    → 其他允许的 .sisyphus/*.md → 放行并记录日志
    }
  }
}
```

零人工干预原则

计划中的所有验收标准必须是 Agent 可执行的命令：

-  `bun test -- src/auth.test.ts` (可执行)
-  `lsp_diagnostics src/auth.ts` (可执行)

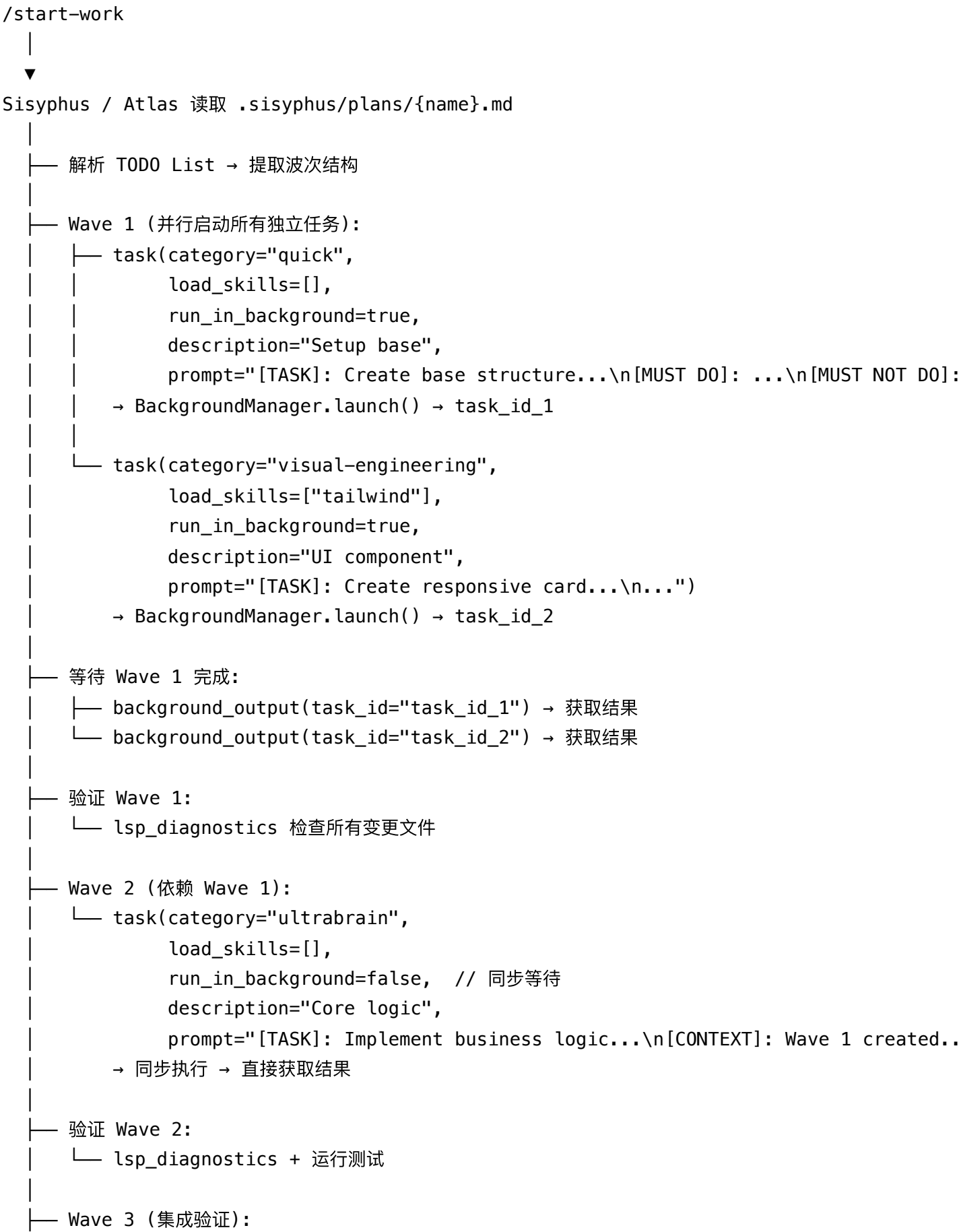
-  "确认 UI 看起来正确" (需要人工)

Build 阶段

触发方式

1. 用户输入 `/start-work` 命令
2. `start-work` hook 处理
3. Sisyphus/Atlas 接管执行

执行流程



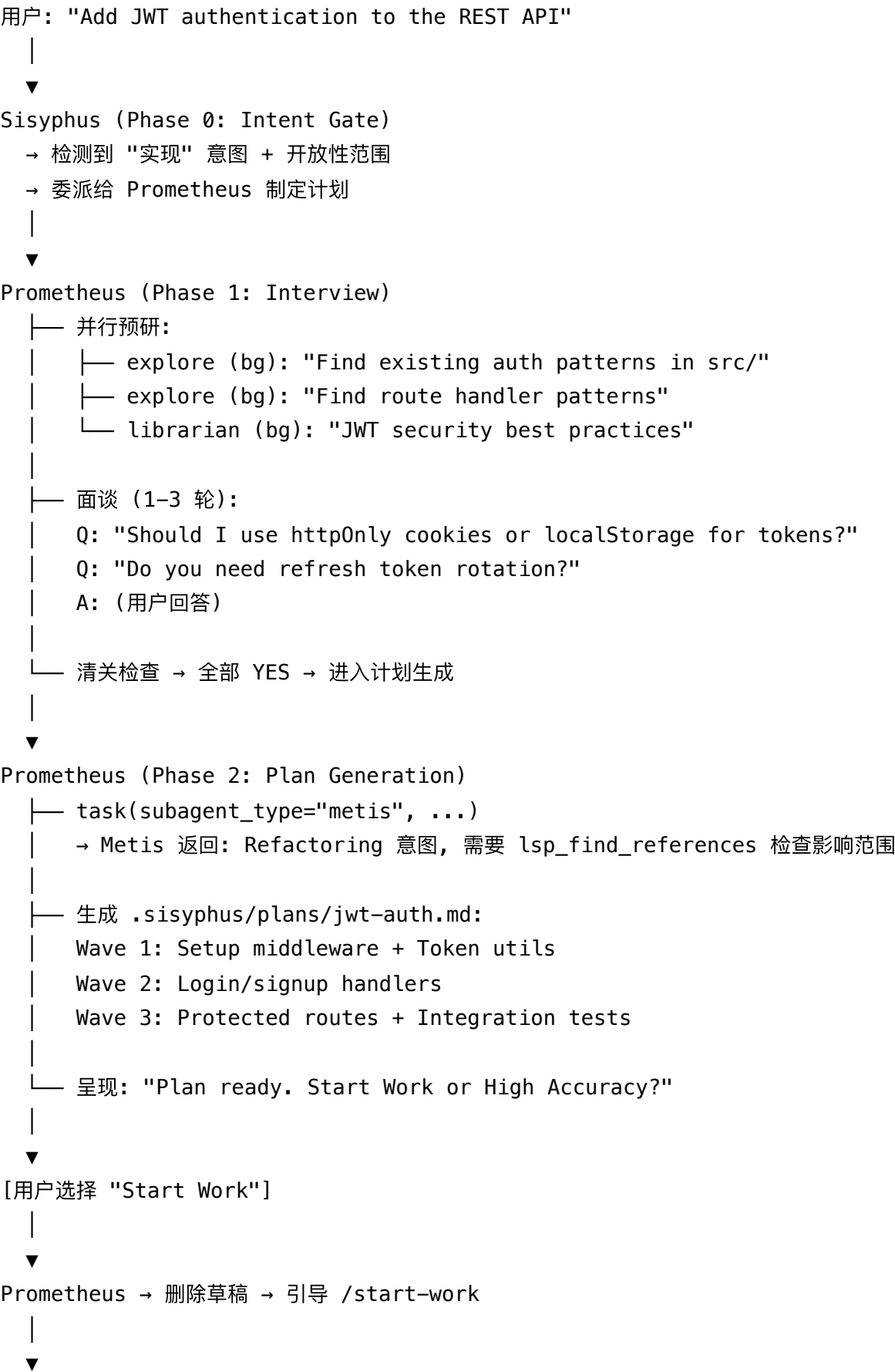
```
| └─ task(category="unspecified-high",
|       load_skills=["testing"],
|       run_in_background=false,
|       description="Integration test",
|       prompt="[TASK]: Run and verify all tests...\n...")
|
└─ 最终验证:
    └─ lsp_diagnostics → 所有文件干净
    └─ 构建通过 (如适用)
    └─ 测试通过 (如适用)
```

失败处理

Task 失败:

```
|
| └─ 使用 session_id 续传修复:
|     task(session_id="ses_xxx",
|           load_skills=[],
|           run_in_background=false,
|           description="Fix: type error",
|           prompt="Fix: TypeError on line 42...")
|
└─ 3 次连续失败:
    1. STOP 所有进一步编辑
    2. REVERT 到最后已知工作状态
    3. DOCUMENT 已尝试方案
    4. 咨询 Oracle
    5. 如 Oracle 无法解决 → 问用户
|
└─ delegate-task-retry Hook:
    自动重试失败的 task 调用
```

端到端完整流程图



Sisyphus 接管 → 读取 .sisyphus/plans/jwt-auth.md

```
|
| └─ Wave 1 (并行):
|   | └─ task(category="quick", skills=[], bg=true)      → "Setup JWT middleware"
|   |   └─ task(category="quick", skills=[], bg=true)    → "Create token utils"
|   |
|   └─ [等待 + 验证 Wave 1]
|
| └─ Wave 2:
|   └─ task(category="ultrabrain", skills=[], bg=false) → "Implement login/signup"
|
| └─ [验证 Wave 2]
|
| └─ Wave 3:
|   └─ task(category="unspecified-high", skills=["testing"], bg=false) → "Integratio
|
| └─ 最终: lsp_diagnostics ✅ + bun test ✅
|       → "JWT authentication implemented and verified."
```

Plan Agent 系统 prompt 注入

源文件: src/tools/delegate-task/constants.ts (580 行) —

buildPlanAgentSystemPrepend() 函数

源文件: src/tools/delegate-task/prompt-builder.ts (55 行) — buildSystemContent() 调用

```

// src/tools/delegate-task/constants.ts - Plan Agent 系统 prompt 构建
export function buildPlanAgentSystemPrepend(
  availableCategories: AvailableCategory[],
  availableSkills: AvailableSkill[]
): string {
  // 注入:
  // 1. 强制上下文收集协议 → 先发 explore/librarian 再规划
  // 2. 可用分类表 (名称 + 描述 + 模型)
  // 3. 可用技能表 (名称 + 描述 + 来源)
  // 4. 任务依赖图模板
  // 5. 并行执行波次模板
  // 6. 分类 + 技能推荐指南
  return PLAN_AGENT_SYSTEM_PREPEND_STATIC_BEFORE_SKILLS
    + buildCategorySection(availableCategories)
    + buildSkillSection(availableSkills)
    + PLAN_AGENT_SYSTEM_PREPEND_STATIC_AFTER_SKILLS
}

export function isPlanAgent(agentName?: string): boolean {
  // 检查是否为 Prometheus/Plan family Agent
}

export function isPlanFamily(agentName?: string): boolean {
  // 广义检查: prometheus, plan, metis, momus
}

// src/tools/delegate-task/prompt-builder.ts - 调用处
export function buildSystemContent(input: BuildSystemContentInput): string | undefined
  const planAgentPrepend = isPlanAgent(agentName)
    ? buildPlanAgentSystemPrepend(availableCategories, availableSkills)
    : ""
  // ...
}

```

防递归:

```

// src/tools/delegate-task/subagent-resolver.ts
if (isPlanFamily(agentName) && isPlanFamily(parentAgent)) {
  return { error: "You are a plan-family agent. You cannot delegate to other plan-famil"
}

```

关键文件索引

文件路径	职责
src/agents/prometheus/system-prompt.ts	Prometheus 系统 prompt 组装
src/agents/prometheus/identity-constraints.ts	身份约束：纯规划者
src/agents/prometheus/interview-mode.ts	Phase 1 面谈逻辑
src/agents/prometheus/plan-generation.ts	Phase 2 计划生成逻辑
src/agents/prometheus/high-accuracy-mode.ts	Phase 3 Momus 审查循环
src/agents/prometheus/plan-template.ts	计划文件 Markdown 模板
src/agents/prometheus/behavioral-summary.ts	行为总结 + 引导 /start-work
src/agents/metis.ts	计划前分析师
src/agents/momus.ts	计划审查员
src/hooks/prometheus-md-only/	Prometheus 文件写入限制
src/hooks/start-work/	/start-work 命令处理
src/tools/delegate-task/prompt-builder.ts	Plan Agent 系统 prompt 注入
src/plugin-handlers/prometheus-agent-config-builder.ts	Prometheus 配置构建
src/plugin-handlers/plan-model-inheritance.ts	Plan → Prometheus 模型继承
src/features/boulder-state/	活跃计划状态追踪